NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *February 9, 2026*

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

Supervisor: Miguel Goulão
Associate Professor, NOVA University Lisbon

Co-supervisor: Grischa Liebel
Associate Professor, Reykjavik University

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *February 9, 2026*

Abstract

Software and conceptual modelling are commonly included in software engineering and related computing programmes, but they often receive less emphasis than other topics in computing education. Students therefore frequently experience modelling as either an essential aid for understanding and communication or as burdensome "extra documentation". Existing research offers valuable insights but is dispersed across different modelling traditions and study contexts. Outcome-focused frameworks (e.g., Bloom-informed competence formulations) clarify what learners should be able to do, while method- and course-specific empirical studies repeatedly report difficulties such as grasping construct meaning, moving from requirements to models, and judging model adequacy. However, there is still limited integrative, empirically grounded explanation of how these challenges arise and how they relate to the strategies and support that students and instructors consider helpful.

This thesis investigates how software modelling is experienced, taught, and learned in higher education, treating modelling as a socio-technical activity that combines conceptual understanding with procedural strategies and is shaped by tools and educational arrangements. The study addresses three research questions: (RQ1) what challenges students and instructors perceive in learning and teaching modelling and how these challenges manifest across modelling activities (e.g., interpreting, creating, refining, evaluating); (RQ2) which strategies are used to address these challenges and what pedagogical and tool-related support needs are identified; and (RQ3) how socio-technical factors such as tools, task domains, feedback practices, and course structures shape modelling experiences.

Methodologically, the thesis adopts a qualitative, theory-building approach based on Grounded Theory. The analysis is conducted with a socio-technical lens inspired by Socio-Technical Grounded Theory to consider how social and technical elements interact. Data are collected through semi-structured interviews complemented by activity-centred elicitation anchored in concrete modelling tasks or artefacts. Bloom's revised taxonomy is used as a sensitising concept to help describe and communicate the cognitive nature of reported challenges, rather than as a coding scheme. The primary outcome is a grounded explanatory account of modelling learning and teaching challenges and associated support

needs; secondarily, the thesis derives implications and design-oriented recommendations for teaching practices and educational tooling aimed at fostering modelling competence in realistic learning contexts.

Keywords: software modelling education · conceptual modelling · grounded theory · socio-technical perspective · modelling competence

Resumo

A modelação de software e a modelação conceptual são frequentemente incluídas em cursos de Engenharia de Software e áreas relacionadas, mas tendem a receber menos destaque do que outros tópicos na educação em Computação. Ainda assim, os estudantes frequentemente percecionam a modelação como um apoio essencial à compreensão e comunicação ou, pelo contrário, como "documentação extra" que compete com o tempo de implementação. A investigação existente em educação para a modelação fornece contributos relevantes, mas encontra-se distribuída por diferentes tradições de modelação e contextos de estudo. Enquadramentos orientados a resultados (por exemplo, formulações de competências informadas pela taxonomia revista de Bloom) ajudam a explicitar o que os estudantes devem ser capazes de fazer, enquanto estudos empíricos centrados em métodos ou unidades curriculares identificam dificuldades recorrentes, como compreender o significado dos construtos, passar de requisitos para modelos e avaliar a adequação dos modelos. Ainda assim, é limitada a explicação integrada e empiricamente fundamentada de como estes desafios emergem e de como se relacionam com estratégias e necessidades de apoio reconhecidas por estudantes e docentes.

Esta tese investiga como a modelação de software é experienciada, ensinada e aprendida no ensino superior, tratando a modelação como uma atividade socio-técnica que combina compreensão conceptual e estratégias procedimentais, e que é moldada por ferramentas e arranjos educativos. O estudo é guiado por três questões de investigação: (RQ1) que desafios estudantes e docentes percecionam na aprendizagem e no ensino da modelação, e como esses desafios se manifestam ao longo de diferentes atividades (por exemplo, interpretar, criar, refinar e avaliar modelos); (RQ2) que estratégias são usadas para lidar com esses desafios e que necessidades de apoio (pedagógicas e relacionadas com ferramentas) são identificadas; e (RQ3) de que forma fatores socio-técnicos, como ferramentas, domínios de tarefa, práticas de feedback e estruturas curriculares, influenciam as experiências de aprendizagem e ensino da modelação.

Do ponto de vista metodológico, a tese adota uma abordagem qualitativa de construção de teoria baseada em Grounded Theory. A análise é conduzida com uma lente socio-técnica, inspirada pela Socio-Technical Grounded Theory, para considerar a interação

entre elementos sociais e técnicos. Os dados são recolhidos através de entrevistas semi-estruturadas, complementadas por elicitação centrada em atividades ancorada em tarefas ou artefactos concretos de modelação. A taxonomia revista de Bloom é usada como conceito sensibilizador para apoiar a descrição e comunicação da natureza cognitiva dos desafios, sem ser aplicada como esquema de codificação. O resultado principal é um relato explicativo fundamentado empiricamente sobre desafios na aprendizagem e ensino da modelação e respetivas necessidades de apoio; de forma secundária, a tese apresenta implicações e recomendações orientadas ao desenho de práticas pedagógicas e de ferramentas educativas que promovam a competência de modelação em contextos realistas.

Palavras-chave: educação em modelação de software · modelação conceptual · grounded theory · perspetiva socio-técnica · competência de modelação

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation

Software systems are complex socio-technical artefacts that must evolve under technical, organisational, and human constraints. In this setting, software and conceptual modelling provide structured abstractions that can support reasoning, communication, and early validation before implementation [5]. In higher education, modelling is therefore commonly taught in software engineering and related programmes through notations and methods such as UML, BPMN, ER, or goal-oriented requirements approaches [22, 21, 16].

Despite its recognised importance, students often perceive modelling in polarised ways: as a valuable instrument for understanding and communicating about systems, or as burdensome "extra documentation" that competes with implementation time [7, 17, 10]. These perceptions are shaped by course-level conditions and by how modelling is situated within the learning workflow—for example, the tools used, the nature of modelling tasks and domains, the role of models in assessment, and how feedback is provided [17, 10]. Understanding how such conditions relate to modelling practice and competence development is important for improving modelling education and for fostering modelling skills that transfer beyond individual course contexts [22].

1.2 Problem statement and research gap

Research on modelling education has produced valuable insights, but evidence and concepts are distributed across different modelling traditions and study contexts [22]. Prior work has (i) proposed outcome-oriented frameworks and taxonomies that help articulate modelling competencies and highlight recurring gaps (e.g., limited explicit attention to evaluation and metacognitive aspects) [3, 2, 4], and (ii) reported, in method- and course-specific empirical studies, recurring difficulties such as grasping construct meaning, moving from requirements to models, and judging model adequacy, often alongside suggestions for pedagogical or tool-mediated support [16, 21].

However, there remains a need for an integrated, empirically grounded understanding

of modelling education challenges and support needs that reflects both student and instructor viewpoints across educational contexts, and that connects reported difficulties to the socio-technical conditions in which modelling is taught and practiced [22, 17].

1.3 Aim and research questions

The aim of this thesis is to develop an empirically grounded account of modelling education challenges and support needs in higher education, treating modelling as a socio-technical activity that combines conceptual understanding, procedural strategies, and contextual influences.

The study is guided by the following research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across different modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

In line with these questions, the analysis focuses respectively on (RQ1) challenges as they appear across modelling activities, (RQ2) coping strategies and articulated support needs, and (RQ3) the socio-technical conditions that shape how modelling is learned and taught.

1.4 Research approach overview

To address these questions, the thesis adopts a qualitative, theory-building approach. The study is conducted using Grounded Theory, informed by a socio-technical stance that attends to interactions between social and technical elements in modelling education [24, 11]. Data are collected from students and instructors through semi-structured interviews, complemented by task- or artefact-anchored elicitation to ground reflections in concrete modelling practice [16]. Bloom's revised taxonomy is used as a sensitising concept to help communicate the cognitive character of reported challenges without constraining the grounded analysis [14, 24]. The primary outcome is a grounded explanatory account of modelling learning and teaching challenges and associated support needs; secondarily, the thesis derives implications and design-oriented recommendations for teaching practice and educational tool support.

1.5 Expected contributions

This thesis contributes to software and conceptual modelling education research in four complementary ways. First, it provides an empirically grounded characterisation of the challenges that students and instructors perceive in learning and teaching modelling, including difficulties that arise across different modelling activities such as interpreting problem descriptions, constructing models, refining them, and judging their quality.

Second, it offers an integrated account of the strategies participants use to cope with these challenges and the support needs they articulate. These support needs span pedagogical practices (e.g., scaffolding, feedback, assessment framing) and tool-mediated support (e.g., guidance, feedback on errors, and mechanisms for managing complexity), and are derived from concrete examples elicited during interviews and activity- or artefact-centred discussion.

Third, the thesis develops a socio-technical explanatory perspective that links modelling challenges and coping strategies to contextual factors such as tool ecosystems, course design choices, feedback practices, and assignment domains. This perspective is intended to clarify how modelling experiences are shaped by interactions between cognitive demands and the educational settings in which modelling is taught and practiced.

Finally, the thesis provides evidence-informed implications for modelling education design. These implications are formulated as actionable recommendations for instructors and course designers and as requirements-level insights for those developing educational modelling tools, with the goal of supporting modelling competence that is robust beyond individual course contexts.

1.6 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides background on software and conceptual modelling, modelling education, grounded theory, Bloom-based learning outcomes, and human factors relevant to modelling learning. Chapter 3 reviews related work on modelling education frameworks, empirical studies of modelling learning challenges, tooling and model-driven engineering in education, and contextual factors such as motivation, inclusion, and authenticity. Chapter 4 presents the research design, research questions, data collection and analysis procedures, and considerations of rigour, ethics, and limitations. Subsequent chapters present the findings and discuss implications for modelling education and tool support.

2

Background

2.1 Software modelling & conceptual modelling

2.1.1 Purpose and role of modelling in software engineering

Software modelling supports reasoning about software systems by creating representations that abstract from reality to emphasise aspects that matter for a given purpose and audience. In software engineering, models can serve multiple roles: they may act as design blueprints, facilitate communication among stakeholders, and enable analysis and validation of properties before implementation [5, 22].

A useful clarification is that modelling should not be reduced to producing diagrammatic pictures. Rather, models are structured artefacts expressed in a modelling language with defined meaning, and their usefulness depends on what they represent and how they are intended to be used. Kühne highlights that the term *model* is often used ambiguously, and proposes a distinction between *type models* (the modelling languages or model types that define what can be expressed) and *token models* (the concrete model instances created for a particular problem) [15]. This distinction helps explain why learning modelling involves more than symbol recognition: it requires understanding the language-level concepts and constraints (type level) and being able to construct, adapt, and assess concrete models for specific tasks (token level).

2.1.2 Conceptual modelling as a broader discipline

It is useful to distinguish between software modelling as a practice within software engineering and conceptual modelling as a broader discipline concerned with representing, structuring, and reasoning about domains and systems across fields. Conceptual modelling is not limited to software development alone; it is also used in organisational analysis, enterprise and service design, data-intensive systems, and other domains where abstraction is needed to manage complexity and support understanding [22]. Reviews of conceptual modelling education underline its interdisciplinary nature and identify a broad

family of modelling subtypes—including data, process, goal, and enterprise modelling—that often evolve in parallel, with limited pedagogical or theoretical integration across subfields [2, 3, 22].

This disciplinary perspective is particularly relevant for research on teaching and learning software modelling. When modelling is treated primarily as a supplementary topic within software engineering courses, students may not fully appreciate its significance and may develop oversimplified views of its purpose and applicability. In contrast, the conceptual modelling perspective frames modelling languages as knowledge schemas: they define what can be expressed, how meaning is structured, and what kinds of analysis or tooling become possible. Understanding modelling therefore requires engaging with notational elements and underlying semantics, as well as modelling goals and the intended audience of the produced model artefacts [5].

2.1.3 Modelling methods, languages, and artefacts

Several authors emphasise that a modelling method is more than a set of symbols or a notation. A modelling method can be understood as comprising a modelling language (including syntax, semantics, and notation), a modelling procedure (the process or steps to create models), and supporting mechanisms and algorithms such as analysis, simulation, or transformation capabilities [4, 5]. This holistic view clarifies why students may be able to reproduce diagrammatic symbols yet still struggle to apply modelling meaningfully: the procedural and analytical aspects of modelling are often less explicitly taught or assessed.

From this viewpoint, the value of a modelling language is not only in enabling representation, but also in supporting reasoning. When a language is treated as a structured schema, models become more than static diagrams: they can become queryable, analysable, and transformable artefacts that enable deeper engagement with a domain. This also motivates tool support that can provide feedback on syntactic correctness, highlight semantic issues, help manage complexity, or automate parts of modelling tasks [2, 22].

2.1.4 Implications for the scope of this thesis

This thesis adopts the view that software modelling is an instantiation of the broader discipline of conceptual modelling. It therefore treats modelling as a discipline involving conceptual understanding of modelling constructs and their semantics, procedural competence in constructing and refining model artefacts from problem descriptions, analytical judgement of model quality and fit-for-purpose, and socio-technical awareness of how modelling practices are shaped by tools, learning contexts, and stakeholder needs. This framing aligns with work on modelling learning outcomes and competencies, which emphasises not only notational knowledge but also procedural skills, evaluative abilities, and reflective understanding [2, 3, 4, 23]. This perspective motivates the subsequent background on modelling education, grounded theory, and learning outcomes, and provides

a foundation for investigating modelling challenges from both student and instructor perspectives.

2.2 Modelling education

2.2.1 Overview of conceptual modelling education

Conceptual and software modelling are widely taught across computing and software-related programmes, but they are taught in diverse ways and appear in different course contexts and curricular roles. Maslov et al. [22] map the conceptual modelling education literature and show that educational work spans multiple modelling sub-communities (e.g., data modelling, process modelling, enterprise modelling), which have developed partially independent emphases, terminology, and pedagogical traditions [22]. Within this landscape, modelling appears in contexts ranging from introductory courses to advanced software engineering modules, and courses emphasise different modelling families and purposes (e.g., data-centric, process-centric, goal-oriented).

The same studies also highlight that much practice in modelling education is shaped by instructor experience and local institutional constraints, as well as by the availability of tools, rather than by systematically articulated educational frameworks or empirically validated design principles [22]. While there is broad consensus on the importance of modelling skills for software professionals, there is less agreement on which specific *competencies* learners should acquire, how these competencies should be scaffolded, and how learning should be assessed in a reliable and educationally meaningful way.

2.2.2 Bloom-based perspectives on modelling learning outcomes

To make modelling learning outcomes more explicit and comparable across courses and contexts, several authors have proposed adaptations of Bloom's revised taxonomy for conceptual and domain modelling. Starting from Bloom's six cognitive process categories—*remember*, *understand*, *apply*, *analyse*, *evaluate*, and *create* [14]—these works operationalise them in modelling terms. Typical modelling-specific interpretations include recalling notation elements and rules (remember), explaining construct meaning and purpose (understand), constructing models by applying modelling rules and procedures to a problem description (apply), checking models for consistency/completeness and identifying defects (analyse), judging model quality and comparing alternatives against criteria (evaluate), and producing or extending models and modelling solutions for new situations (create).

Bogdanova and Snoeck analyse learning outcomes and assessment tasks for domain modelling through the lens of Bloom's revised taxonomy, reporting that many tasks concentrate on *understanding*, *analysing*, and *creating conceptual knowledge*, while giving less explicit attention to *evaluation* activities and to *procedural* and *metacognitive* knowledge [3]. Notably, across Bloom-informed modelling frameworks the mapping between

Bloom categories, modelling tasks, and knowledge types is not uniform (some works emphasise cognitive processes, others additionally foreground the knowledge dimension); however, a recurring observation is the relative underrepresentation of evaluation and reflective/metacognitive aspects in current educational practice.

Bork extends this Bloom-based perspective to both conceptual modelling and metamodeling, mapping knowledge and cognitive dimensions to all components of a modelling method (language, semantics, notation, procedure, mechanisms and algorithms) [4]. An evaluation of a "Smart City" teaching case using this framework reveals that even rich, practice-oriented material may still underemphasise certain cognitive levels (e.g., evaluation) or knowledge types (e.g., metacognitive knowledge). These works collectively suggest that Bloom's taxonomy can provide a useful lens for reflecting on and improving modelling education, by making learning objectives more explicit and balanced across different dimensions.

2.2.3 Process and goal modelling education

Beyond data and structural modelling, there is growing interest in how students learn process and goal-oriented modelling. For business process modelling with BPMN, Maslov outlines a research agenda for developing empirically validated process-modelling education that integrates insights from conceptual modelling education, process model quality research, and learning sciences [21]. This agenda highlights the need for systematic mapping of existing work, empirical studies of novice modellers' behaviour and typical errors, and the design and experimental evaluation of BPMN-specific teaching strategies and feedback mechanisms. This reflects a broader recognition that teaching process modelling effectively requires more than just introducing notation; it requires attention to cognitive demands, quality criteria, and the kinds of feedback needed to help learners improve.

Goal-oriented requirements modelling approaches introduce additional layers of complexity, as it requires learners to reason about stakeholder intentions, rationales, and dependencies. An iStar learning study explicitly investigates the challenges students face when learning iStar, using a combination of Bloom-based task framing and grounded theory analysis of tutorials, modelling sessions, and interviews [16]. The authors report difficulties at multiple cognitive levels, including understanding the semantics of iStar constructs, applying the notation to real requirements, and analysing models to assess completeness and appropriateness. They derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, visualisation, and semi-automation modelling from text) that could help address these challenges [16].

These strands of work on BPMN and iStar education illustrate that different modelling families pose distinct learning challenges and may require specialised pedagogical approaches. They also reinforce broader themes in modelling education, including the need for scaffolding from text to model, support for managing complexity, and explicit

attention to how learners evaluate and refine their models.

2.2.4 Modelling, tools, and model-driven engineering in education

Modelling in educational settings is often mediated by tools, ranging from simple drawing tools and web-based modellers to industrial-grade model-driven engineering (MDE) environments. Tool choice can influence students' perceptions of modelling and their opportunities to practice particular skills. Liebel et al. conducted a multi-case study of model-driven software engineering courses that use different tools (e.g., Umple, Papyrus) and place different emphasis on model-driven architecture and code generation [17]. Their results indicate that students' perceptions of tools are highly context-dependent: the same tool may be perceived positively in one course setup and negatively in another, depending on project design, available support, and the roles that models play (e.g., for communication versus strict code generation). They also find that perceptions of UML as modelling language are more positive and robust when models are used for requirements and analysis, rather than only as precise inputs to code generators [17].

These findings align with broader discussions of human factors in model-driven engineering, which argue that educational tools should prioritise usability, cognitive support, constructive feedback, and alignment with learning goals over industrial feature completeness [18]. From an educational perspective, this suggests that tool selection and integration should be carefully considered to ensure they support the intended modelling competencies and do not inadvertently increase cognitive load or distract from learning objectives.

2.2.5 Competence-oriented and human-centred perspectives

Broader software engineering education research brings in complementary competence-oriented and human-centred perspectives that can inform modelling education. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, arguing that curricula should be grounded in explicit competence models rather than only topic lists [23]. They introduce a six-level classification system (remember, understand, explain, use, apply, develop) for technical competencies and outline an iterative, data-driven approach to relating teaching methods and structural conditions to competency development [23]. This approach resonates with the Bloom-based frameworks discussed earlier, but emphasises the need for empirical validation and alignment with real-world professional requirements.

Work on human factors in modelling and software engineering education further highlights the role of motivation, inclusion, and authenticity. For example, studies on domain diversity and motivation in modelling courses show that educators' assumptions about which domains are motivating (e.g., games, automation) do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [10]. Students also report that feeling excluded by

unfamiliar or culturally distant domains, or by assignment setups that do not fit their backgrounds, can hinder their engagement and learning [10]. More generally, experiential education initiatives emphasise the value of working on real systems, real tasks, and with real mentors to bridge the gap between classroom modelling exercises and the complexities of practice [12].

Taken together, these competence-oriented and human-centred perspectives reinforce the idea that modelling education should be designed not only around notations and tools, but also around the competencies to be developed, the diversity of learners, and the authenticity and inclusiveness of modelling tasks and domains.

2.2.6 Implications for this thesis

The literature on modelling education, Bloom-based frameworks, process and goal modelling, tooling, competencies, and human factors provides several important implications for this thesis. First, it suggests that modelling learning outcomes can be systematically described in terms of cognitive and knowledge dimensions, and that existing courses may underemphasise evaluation and metacognitive aspects of modelling competence [2, 3, 4]. Second, it shows that different modelling families (e.g., data, process, goal) can pose partly distinct learning demands and error patterns, meaning that findings are not automatically transferable across notations and tasks. At the same time, family-specific empirical studies—such as the iStar learning work—provide a useful methodological precedent for how to elicit challenges in a task-anchored way and derive support requirements grounded in learner experience, which this thesis adapts to a broader educational scope. [16, 21]. Third, it highlights that tools and model-driven approaches can both enable and hinder learning, depending on how they are integrated into course designs and on the kind of feedback and affordances they provide [17, 18]. Finally, competence-oriented and human-centred research indicates that modelling education must be sensitive to students' motivations, backgrounds, and learning environments, and that explicit competence models can support more systematic design and evaluation of teaching [23, 12, 7].

These insights inform the research questions and methodological choices of this thesis, which aims to investigate how students and teachers experience modelling, which challenges they perceive, and what kinds of support may foster the development of modelling competencies in realistic educational contexts.

2.3 Grounded Theory

2.3.1 Overview and key characteristics

Grounded Theory (GT) is a qualitative research methodology originally developed in sociology with the aim of generating theory that is systematically grounded in data rather than deduced from prior hypotheses. Instead of starting from a fixed theoretical

framework, GT proceeds iteratively: data collection and analysis occur simultaneously, and emerging insights guide subsequent sampling and questioning. Core practices include *coding* (assigning conceptual labels to segments of data), *constant comparison* (comparing new data with existing codes and categories), and *theoretical sampling* (collecting new data to refine or challenge emerging concepts), and extensive *memoing* to capture analytic ideas [24, 1].

The outcome of a GT study is not a set of descriptive themes, but a more abstract, integrated explanation of how participants handle a central problem or concern. This explanation is typically organised around a *core category* that captures a dominant pattern in the data and links other categories together. Evaluation criteria emphasise the *fit* of concepts to the data, the *workability* of the emerging theory in explaining the studied phenomenon, its *relevance* to practitioners and researchers, and its *modifiability* in light of new data, rather than statistical notions of validity [1].

Over time, GT has evolved into several variants, including *Classic GT* (Glaser and Strauss), *Straussian GT* (Strauss and Corbin), and *Constructivist GT* (Charmaz) [9, 25, 8]. While these variants share core principles, they differ in the degree to which they prescribe specific coding procedures, the role of prior theory, and the epistemological stance of the researcher. For software engineering research, this diversity offers flexibility but also introduces risks of inconsistent or superficial use if methodological choices are not made explicit [24].

2.3.2 Grounded Theory in software engineering

In software engineering (SE), GT has been used to study topics that are difficult to capture through controlled experiments or surveys, such as developer experience, team practices, decision-making processes, and socio-technical phenomena. A critical review by Stol et al. [24] finds that GT has become increasingly popular in SE research, but that many studies lack methodological rigour, adopt only parts of GT procedures, and show limited reflexivity regarding the researcher's role [24]. To address these issues, they offer a set of guidelines on when GT is appropriate (e.g., for under-theorised, practice-oriented problems), how to choose among GT variants, and how to report GT studies in a way that allows readers to assess their quality.

Adolph et al. provide a concrete example of applying classical (Glaserian) GT to study how software practitioners cope with the process of development, while also reflecting on the practical challenges of conducting GT research in SE contexts [1]. They describe in detail how they conducted open coding on interviews and field observations, developed concepts and categories through constant comparison, and iteratively refined their sampling strategy to explore interesting phenomena in more depth. Their work highlights the importance of staying close to the data, avoiding overly mechanical use of qualitative analysis tools, and systematically documenting analytic decisions through memos.

These contributions establish GT as a suitable methodology for SE when the research goal is to generate mid-level substantive theory about human and organisational aspects of software development, and they provide concrete guidance for designing and reporting such studies.

2.3.3 Socio-Technical Grounded Theory

Hoda et al. propose Socio-Technical Grounded Theory (STGT) as a specialisation of GT for SE that explicitly foregrounds the socio-technical nature of software development [11]. STGT retains the core principles of GT (iterative data collection and analysis, constant comparison, theoretical sampling, memoing) but emphasises the need to attend simultaneously to social and technical elements and their interactions. This involves, for example, paying attention to roles, norms, tools, artefacts, and organisational structures, and analysing how these co-evolve over time.

STGT also articulates expectations for rigour in terms of credibility, transferability, dependability, and confirmability, and provides detailed guidance on reporting, including the description of research context, participants, data sources, coding procedures, and the evolution of categories and core concepts [11]. Therefore, STGT offers SE researchers a reporting template tailored to the realities of socio-technical systems.

2.3.4 Implications for this thesis

This thesis investigates how students and teachers experience software and conceptual modelling, and how they perceive challenges, learning processes, and support needs. These are socio-technical questions that involve individual cognition, educational practices, tools, and organisational constraints, and they are not yet well theorised in the software modelling education literature. Grounded Theory is therefore an appropriate methodological choice for this work.

2.4 Bloom's Taxonomy and learning outcomes

2.4.1 Overview of Bloom's revised taxonomy

The original Bloom's taxonomy was presented as a hierarchical classification of educational objectives, describing increasingly complex cognitive activities from simple recall to higher-order thinking. Krathwohl's revision restructures the taxonomy into a two-dimensional framework that distinguishes between a *knowledge dimension* (factual, conceptual, procedural, metacognitive) and a *cognitive process dimension* (remember, understand, apply, analyse, evaluate, create) [14]. This revised taxonomy encourages educators to formulate learning outcomes that are explicit both about the type of knowledge involved and the level of cognitive processing expected from learners.

In the revised framework, *factual knowledge* concerns basic elements that students must know, such as terminology and specific details; *conceptual knowledge* involves relationships among elements and larger structures; *procedural knowledge* relates to methods, techniques, and criteria for using procedures; and *metacognitive knowledge* refers to awareness of one's own cognition and learning strategies. The cognitive process dimension ranges from lower-order processes such as remembering and understanding, through applying and analysing, to higher-order processes of evaluating and creating [14]. The combination of these two dimensions supports a more fine-grained articulation of what learners should know and be able to do.

2.4.2 Learning outcomes in computing and software engineering

In computing and software engineering education, Bloom's revised taxonomy is widely used to express learning outcomes and to align teaching and assessment with intended levels of cognitive demand [14, 6]. Typical applications include mapping course objectives to cognition levels, ensuring that programmes do not over-concentrate on lower-order outcomes, and checking that assessment tasks actually require the intended level of performance. However, generic formulations of the taxonomy can be difficult to apply directly in specialised domains such as software modelling, where domain-specific activities (e.g., interpreting a diagram, constructing a model from text, critiquing a modelling method) require more tailored descriptions.

This has led to several domain-specific adaptations of Bloom's taxonomy in areas such as domain modelling, conceptual data modelling, and metamodeling [3, 2, 4]. These adaptations retain the underlying cognitive-process levels while refining the knowledge categories and example tasks to reflect the particular demands of modelling work.

2.4.3 Bloom-based formulations of modelling learning outcomes

Several works in the modelling education literature explicitly build on Bloom's revised taxonomy to describe and analyse modelling-related learning outcomes. Bogdanova and Snoeck adapt the taxonomy to domain modelling by specifying what each cognitive level can mean in this context, such as remembering notation elements, understanding the meaning of relationships, applying modelling rules to construct diagrams, analysing models for completeness and consistency, evaluating alternative designs, and creating new models from complex descriptions [3]. Using this adapted taxonomy, they classify a large set of assessment tasks and show that existing practice tends to focus on understanding, analysing, and creating conceptual knowledge, with comparatively little emphasis on evaluation tasks, procedural knowledge, or metacognitive learning outcomes [3].

Extending this work, CaMeLOT proposes a three-dimensional framework for conceptual data modelling that explicitly combines Bloom's cognitive levels with the knowledge dimension and specific content areas such as classes, relationships, constraints, modelling methods, quality criteria, and tool usage [2]. The authors instantiate this framework

with example learning outcomes, demonstrating how it can be used to design, review, or compare modelling courses. Their analysis also suggests that metacognitive outcomes (e.g., reflecting on modelling strategies, recognising one's own limitations) are rarely made explicit in current curricula [2].

Bork further generalises the Bloom-based perspective by developing separate yet related taxonomies for conceptual modelling and metamodeling [4]. He maps the knowledge and cognitive dimensions to the components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) and uses this mapping to evaluate a Smart City teaching case. The evaluation reveals an imbalance: many activities target creation and application at higher cognitive levels, while explicit evaluate-level tasks and metacognitive activities are largely absent [4]. This pattern is consistent with the findings of Bogdanova and Snoeck and suggests that modelling education often emphasises the ability to produce models but gives less structured attention to helping students judge model quality or reflect on their modelling practice.

These Bloom-based formulations show that learning outcomes for modelling can be described systematically across different levels and types of knowledge, and they provide concrete examples of how to manage this in educational settings. They also highlight recurring gaps, particularly in relation to evaluative and metacognitive aspects of modelling competence.

2.4.4 Bloom, competencies, and this thesis

For this thesis, Bloom's revised taxonomy and its modelling-specific adaptations provide a conceptual vocabulary for describing modelling competence in terms of cognitive demands and knowledge types. In particular, they help distinguish challenges related to understanding modelling constructs and their semantics, applying modelling procedures to construct and refine model artefacts, and exercising evaluative judgement when assessing model quality and fit-for-purpose. The recurring gaps identified in prior work—especially the limited explicit emphasis on evaluation and metacognitive outcomes in many course designs—motivate a closer empirical examination of how students and teachers experience modelling and what kinds of support they perceive as needed [3, 2, 4, 23].

2.5 Human factors in modelling and SE education

2.5.1 Motivation, engagement, and the "value" of modelling

Software and conceptual modelling are often perceived by students as either highly valuable (e.g., for understanding complex systems and communicating design intent) or as burdensome "extra documentation" that competes with implementation time [10]. Such perceptions are not merely a matter of individual preference; they reflect a broader set of human factors that shape engagement, learning effort, and ultimately competence

development. In modelling education, learners frequently report that motivation depends on whether tasks feel authentic, whether modelling outputs matter beyond grading, and whether modelling is integrated into a meaningful workflow rather than treated as a stand-alone academic exercise. [10, 12]

Research on modelling education explicitly highlights motivation as a key factor. A study on domain diversity, motivation, and inclusion in software modelling education shows that educators' assumptions about which domains are motivating do not always match students' preferences [10]. Students tend to value domains that connect to their interests, everyday experiences, or socially relevant contexts, and they report higher motivation when they have some agency (e.g., choice or personalisation of the domain) [10]. These results suggest that human factors in modelling education cannot be separated from how modelling tasks are framed, which domains they use, and whether students can see a clear purpose for modelling beyond producing a diagram.

2.5.2 Inclusion, domain diversity, and participation barriers

Human factors also include issues of inclusion and participation. Even when modelling topics are formally accessible, examples and assignments can implicitly privilege specific backgrounds, interests, or prior experiences. Domain-diversity research indicates that certain commonly used modelling domains (e.g., video games or specific technical automation scenarios) may be engaging for some students but alienating for others [10]. Students may also experience barriers related to group-work structures, competitive gamification elements, or assumptions about familiarity with particular cultural or socioeconomic contexts. These factors can influence not only motivation but also opportunities for practice and feedback, thereby affecting learning outcomes.

From a course-design perspective, these findings motivate the careful selection of modelling domains and assignment setups, balancing instructors' preferences and curricular relevance with the diversity of student experiences and interests. Providing alternative domains, allowing domain choice, and ensuring that examples do not rely on narrow cultural references can be considered human-centred strategies to improve participation and reduce disengagement [10].

2.5.3 Tools as mediators of learning experience

In modelling education, tools play a critical role as mediators of the learning experience. They actively shape how students interact with modelling concepts, including how easily they can experiment, receive feedback, manage complexity, and recover from errors. Usability issues, steep learning curves, or misalignment between tool features and learning objectives can lead to frustration and negative attitudes toward modelling, even when the underlying notation is conceptually useful.

Liebel et al. examine model-driven software engineering education through a multi-case study and report that students' perceptions of modelling tools are highly context-dependent [17]. The same tool may be perceived as useful or cumbersome depending on how it is integrated into the course design, what the modelling artefacts are used for (e.g., communication vs. code generation), and the level of support provided. They also find that students tend to have more positive perceptions of UML when models are used for analysis and communication, even if tool-related issues reduce satisfaction in code-generation contexts [17]. This suggests that negative experiences are frequently tied to the socio-technical configuration of the course (tool, task, expectations) rather than to modelling as a concept.

2.5.4 Authenticity, experiential learning, and social dynamics

Human factors in SE education also encompass the extent to which learning activities resemble real-world practice. Experiential education initiatives emphasise that authenticity can enhance motivation, engagement, strengthen soft-skill development, and improve students' understanding of the complexities of software practice. Holmes et al. identify three "dimensions of experientialism" in a large, distributed capstone programme: working on real projects (existing open-source systems and codebases), performing real tasks (non-trivial contributions), and engaging with real mentors (experienced developers embedded in the project) [12]. Students report that this combination helps them foster communication skills, teamwork, time management, and practical problem-solving abilities, while also deepening their technical understanding in authentic contexts [12]. Although modelling courses may not always be structured as capstones, the experientialism perspective is still relevant.

2.5.5 Competence development and human-centred educational design

Competence-oriented research in SE education argues that curricula should be grounded in explicit competence models of what students should be able to do, and that teaching methods and structural conditions should be linked to competence outcomes rather than treated as ad hoc choices [23]. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, introducing a multi-level classification system for technical competencies intended to describe progression from basic familiarity to the ability to develop new solutions [23]. This approach emphasises iterative improvement through measuring competence levels, identifying influencing factors (structural conditions and process variables), and refining course designs accordingly [23].

Adopting such a competence-oriented and human-centred perspective in modelling education may require more than selecting an engaging domain or user-friendly tool; it may also involve designing learning activities and assessments that support the systematic progression across competencies, including conceptual understanding, procedural ability

to create and refine models, evaluative judgement of model quality, and reflective awareness of modelling choices and limitations. The reviewed literature indicates evaluation and metacognitive competencies are often underemphasised in current modelling education [3, 2, 4], and human factors research provides a complementary explanation: learners may disengage when feedback is insufficient, tasks feel artificial, tools are frustrating, or domains do not resonate with their interests and experiences.

2.5.6 Implications for this thesis

The human factors literature motivates this thesis in several ways. First, it highlights that modelling education should be analysed not only in terms of notations and learning outcomes, but also in terms of student and teacher experiences shaped by motivation, inclusion, feedback, and perceived authenticity [10, 12]. Second, modelling tools should be treated as socio-technical mediators that influence cognitive load, engagement, and attitudes toward modelling, making tool choice and integration important pedagogical determinants of learning experience [17, 18]. Third, competence-oriented perspectives suggest that empirical investigation should seek to connect perceived challenges and support needs to explicit competence development and course design choices [23].

3

Related Work

3.1 Conceptual modelling education landscape

Research on conceptual modelling education spans multiple modelling families and research communities, including data modelling, process modelling, requirements-related modelling, and model-driven development. Maslov et al. map the conceptual modelling education literature and report that educational work is distributed across multiple communities and modelling subfields that have developed partially independent terminology, learning-theory grounding, and evaluation approaches [22]. As a result, findings are often difficult to compare across modelling families, and evidence about effective teaching strategies and supports accumulates unevenly across subdomains.

This thesis builds on the landscape evidence by drawing together frameworks and empirical results from different modelling-education strands (conceptual/data, process, goal-oriented, and tool-supported modelling) and by focusing on the challenges and support needs reported by both students and instructors.

3.2 Modelling frameworks

A common approach to strengthening modelling education research is to define explicit frameworks for learning outcomes, competencies, and assessment design. Several works adapt Bloom's revised taxonomy [14] to modelling activities in order to clarify what learners should know and be able to do beyond learning a notation. The following subsections summarise three modelling-education frameworks that are especially relevant to this thesis.

3.2.1 CaMeLOT: a framework for conceptual data modelling learning outcomes

CaMeLOT proposes an educational framework for conceptual data modelling that combines Bloom's cognitive levels with a knowledge dimension (factual, conceptual, procedural, metacognitive) and modelling content areas [2]. The framework is instantiated with

detailed example learning outcomes and can be used to design course objectives, learning activities, and assessments in a systematic manner. For this thesis, CaMeLOT is valuable as a template for articulating modelling competence across cognitive and knowledge dimensions, and for motivating the inclusion of procedural and reflective elements rather than focusing exclusively on diagram production.

3.2.2 Domain Modelling in Bloom: empirical analysis of assessment practices

Bogdanova and Snoeck empirically analyse domain-modelling assessment tasks through the lens of Bloom's revised taxonomy [3]. They report that many tasks concentrate on understanding, analysing, and creating conceptual knowledge, while evaluation, procedural knowledge, and metacognitive learning outcomes receive limited explicit attention. This work provides evidence that modelling courses may implicitly expect higher-level modelling competence without systematically scaffolding or assessing foundational skills. For this thesis, these findings support examining modelling challenges in relation to cognitive demands and considering whether educational support should target not only syntax and notation use but also strategies for evaluation and refinement.

3.2.3 Teaching conceptual modelling and metamodeling

Bork extends Bloom-based framing to both conceptual modelling and metamodeling by relating cognitive and knowledge dimensions to components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) [4]. By analysing a teaching case using the proposed framework, the work suggests that even practice-oriented materials may emphasise model creation while providing limited explicit support for evaluation tasks and metacognitive reflection. This perspective is useful for the present thesis because it frames modelling competence as method-level understanding and reflective judgement, aligning with the thesis focus on the transition from notation use to meaningful modelling practice.

3.3 Empirical studies on modelling learning challenges

Frameworks clarify educational targets, but empirical studies are needed to understand where learners struggle in practice and which supports are perceived as valuable. Two complementary strands are particularly relevant: studies of goal-oriented modelling learning challenges and research agendas for process modelling education.

3.3.1 iStar learning challenges and support requirements

A key empirical reference for this thesis is an iStar learning study, which investigates challenges beginners face when learning and practising iStar and derives requirements for facilitating learning [16]. Using Bloom-based task framing and grounded theory analysis,

the study reports difficulties at multiple cognitive levels, including understanding the semantics of modelling constructs, applying the notation to requirements scenarios, and analysing models for quality and completeness. The authors derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, advanced visualisation, and partial automation from textual requirements) and implement a subset of these requirements in a prototype tool [16]. This thesis builds on these insights by extending the focus from a single modelling method to broader modelling education contexts and by eliciting perspectives from both students and instructors.

3.3.2 Process modelling education and the need for empirical validation

In the domain of process modelling, Maslov argues for empirically validated process-modelling education using BPMN and outlines a research agenda that integrates conceptual modelling education, process model quality research, and learning theory [21]. The agenda emphasises systematic synthesis of existing work, empirical investigation of novice modelling behaviour and typical errors, and the design and experimental validation of BPMN-specific teaching strategies and feedback mechanisms. This agenda is relevant to the present thesis as a parallel strand: it highlights that modelling families pose distinct cognitive and practical demands while pointing to recurring educational needs such as scaffolding, feedback, and support for managing modelling complexity.

3.4 Tools and model-driven engineering in education

Modelling education is strongly shaped by tool choice, especially in model-driven and model-based engineering settings. Tools can support learners through feedback, automation, and manageability, but they can also introduce friction that influences how students perceive modelling tasks and their own competence.

3.4.1 Perception of tools and UML in MDE education

Liebel et al. show that students' tool experiences depend less on the tool "in isolation" than on how it is embedded in the course—including the support provided, the surrounding workflow, and the expectations placed on models (for example, exploratory communication artefacts versus precise inputs to downstream automation) [17]. The study also suggests that perceptions of UML are more robustly positive when models are used for requirements and analysis rather than primarily as code generation inputs [17]. These results motivate treating tooling and course design as socio-technical factors that shape modelling experiences, rather than as neutral delivery mechanisms.

3.4.2 Human factors in model-driven engineering

Work on human factors in model-driven engineering argues for research that accounts for usability, cognitive load, and the alignment between tools and user goals [18]. From an educational perspective, this supports the view that modelling tools should be evaluated not only by feature completeness, but also by how they support learning processes such as exploration, error detection, reflection, and progressive mastery.

3.5 Human factors and learning context in modelling education

Beyond frameworks, methods, and tools, modelling education is influenced by motivational and contextual factors such as domain choice, inclusiveness, and the authenticity of tasks.

3.5.1 Domain diversity, motivation, and inclusion

Research on domain diversity and motivation in modelling education reports that educators' assumptions about "motivating" domains do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [10]. The work also highlights that assignment design can unintentionally exclude learners when domains or examples presume cultural, economic, or experiential background that not all students share [10]. These findings motivate treating domain choice and inclusiveness as relevant contextual variables when interpreting modelling learning experiences.

3.5.2 Experiential learning and authenticity

Holmes et al. propose dimensions of experientialism in software engineering education and show that students value learning experiences that involve real projects, real tasks, and real mentors [12]. Although this work is not modelling-specific, it contributes a useful lens for interpreting how authenticity and mentorship can influence learners' engagement and competence development, and it supports the relevance of studying modelling experiences in realistic educational contexts.

3.6 Competence-oriented agendas and methodological precedents

Competence-oriented perspectives provide a complementary lens for structuring educational goals and for linking teaching approaches to measurable outcomes.

3.6.1 Competency identification and development in SE education

Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing competencies in software engineering and argue that curricula should be grounded in explicit competence models rather than topic lists [23]. They introduce a six-level classification system for technical competencies and advocate an iterative approach in which competence targets are identified, competence levels are measured, and teaching strategies and structural constraints are adjusted over time [23]. While not specific to modelling, this agenda supports treating modelling competence as multi-dimensional (conceptual, procedural, analytical, socio-technical) and complements Bloom-based modelling frameworks by emphasising explicit competence models and iterative alignment between competence targets, teaching methods, and structural constraints.

3.6.2 Grounded Theory in software engineering research

Methodologically, grounded theory has been widely discussed in software engineering research as an approach for developing explanatory theory from qualitative data. Stol et al. provide a critical review and practical guidelines for conducting grounded theory in software engineering [24], while Hoda et al. propose a socio-technical grounded theory perspective that explicitly accounts for the interplay between social and technical factors [11]. Adolph et al. offer a practical account of using grounded theory to study the experience of software development [1]. These works provide methodological grounding for the approach used in this thesis and support the use of interviews and iterative coding to derive an explanatory account from participant experiences.

3.7 Summary

In summary, existing work provides: (i) landscape-level evidence that modelling education research is distributed across modelling families and communities [22]; (ii) frameworks that structure modelling learning outcomes using Bloom-based dimensions [3, 2, 4]; (iii) empirical studies and agendas that identify modelling-specific learning challenges and the need for validated educational support [16, 21]; and (iv) tool-, competence-, and human-centred perspectives that emphasise the role of context, usability, motivation, inclusiveness, and authenticity [17, 10, 12, 23].

4

Methodology

4.1 Research design

This study uses a qualitative, theory-building research design based on Grounded Theory (GT) as adapted for software engineering research [24, 1]. To explicitly attend to the interplay between social and technical elements in modelling education (e.g., students and instructors, teaching practices, notations, tools, and artefacts), the analysis is informed by Socio-Technical Grounded Theory (STGT) [11]. Data collection and analysis proceed iteratively, allowing emerging insights to shape subsequent sampling and probing [1, 24].

The overall design is informed by prior modelling-education research that combines modelling activities with interviews to elicit situated reasoning and derive support requirements, particularly the iStar learning study [16]. The present work applies a similar activity-anchored elicitation logic while targeting a broader range of modelling contexts beyond a single modelling method.

4.2 Research questions and operational focus

The study is guided by three research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

To guide data collection and analysis, each research question is treated as an analytic focus rather than a hypothesis to test. RQ1 focuses on characterising challenges and

their variation across modelling activities and contexts. RQ2 focuses on coping strategies and articulated support needs, including the conditions under which particular forms of support are seen as helpful. RQ3 focuses on linking challenges and strategies to contextual influences, clarifying how social and technical factors co-shape modelling experiences.

4.3 Study context and participants

The study is conducted in higher education software engineering and related courses that include software modelling components (e.g., requirements engineering, software design, business process modelling, or model-driven engineering). Two participant groups are targeted:

- **Students:** learners who have encountered modelling activities in at least one course and have completed at least one modelling-related assignment or project.

- **Instructors:** lecturers or teaching assistants who have taught modelling-related content and have experience assessing student models or guiding modelling activities.

Sampling follows the principle of theoretical sampling: initial participants are recruited to seed the analysis, and subsequent recruitment is guided by emerging categories (e.g., variation across courses, notations, tool ecosystems, or assignment styles) until categories are sufficiently elaborated [1, 24]. Inclusion criteria emphasise direct experience with modelling tasks in order to support experience-near accounts of challenges, strategies, and contextual constraints.

4.4 Data collection

Data collection is based on semi-structured interviews with two stakeholder groups: (i) students who have recently engaged in modelling tasks in higher-education courses, and (ii) instructors (including lecturers and teaching assistants) who teach or assess modelling-related work. Two interview guides were prepared—one per group—with a shared backbone (perceptions of modelling, challenges, strategies, and support needs) but different emphases: student interviews focus on learning experiences and task-level difficulties, while instructor interviews focus on teaching goals, scaffolding, assessment, and observed misconceptions. Interviews typically last 20–30 minutes for both groups, and are complemented by an activity-centred elicitation component anchored in a concrete modelling task or artefact to ground reflections in situated examples of modelling practice, following the logic used in prior modelling education research [16]. The procedure was refined through a pilot to validate interview flow, estimate timing, and improve probing for concrete examples.

4.4.1 Interview guides and pilot

Two semi-structured interview guides were developed: one for students and one for instructors. The guides share a common backbone (perceptions of modelling, challenges, strategies, support needs) but differ in emphasis. Student interviews focus on learning experiences and task-level difficulties; instructor interviews focus on teaching goals, scaffolding and assessment practices, and observed student misconceptions.

The pilot interview phase was used to refine question wording, ordering, and probing strategies, with the goal of improving the specificity of examples and the clarity of contextual detail. Bloom-based modelling frameworks informed the formulation of prompts by mapping modelling competence to observable activities (e.g., interpreting models, constructing models from text, and judging model quality), supporting broad coverage of cognitive demands without imposing Bloom as a coding template.

4.4.2 Activity-centered elicitation

To move beyond abstract opinions, participants are invited to discuss a concrete modelling task or artefact. Depending on participant context, this includes one of the following: (i) a short modelling exercise based on a brief scenario, or (ii) a retrospective walkthrough of a recent modelling assignment or model created/assessed in a course.

This activity-centered elicitation is inspired by the iStar learning study, which used a tutorial and modelling session followed by an interview to anchor reflections in a concrete modelling experience [16]. The goal is to surface tacit reasoning, decision points, typical errors, and tool- or feedback-related constraints that may not emerge from interview questions alone.

4.4.3 Interviews

Interviews are conducted individually and last approximately 20–30 minutes. Student interviews cover background, perceived usefulness of modelling, learning difficulties, strategies for going from text to model, evaluation criteria, tool experiences, and motivational and contextual factors (including inclusion-related experiences where relevant). Instructor interviews address teaching objectives, scaffolding practices, assessment design, common student misconceptions, tool choices and constraints, and perceived gaps in current pedagogy. Interviews include flexible probing to explore emerging themes in depth, consistent with GT's iterative and adaptive logic [1, 24].

4.5 Data analysis

4.5.1 Grounded Theory variant and socio-technical stance

The analysis follows Grounded Theory principles as adapted for software engineering research [24, 1]. In particular, the study adopts a socio-technical stance aligned with

STGT, treating modelling education as an interaction between social elements (students, instructors, feedback practices, course constraints) and technical elements (notations, tools, model artefacts, automation features) [11]. This stance informs both the coding process and the interpretation of emerging categories.

4.5.2 Coding process

The coding process follows an iterative progression of open coding, axial coding, and selective coding. Open coding labels segments of interview transcripts and artefacts with conceptual codes capturing modelling challenges, strategies, perceptions of value, tool interactions, and contextual constraints. Constant comparison is applied throughout: newly coded incidents are compared to previous incidents and codes to refine categories and identify variation [1].

Axial coding relates categories using a coding paradigm (e.g., phenomenon, causal conditions, context, intervening conditions, and strategies), as demonstrated in modelling education research on iStar learning [16]. Selective coding then integrates categories into a coherent explanatory account. Memoing is used continuously to document analytic decisions, emerging hypotheses, and links between categories [24, 1].

4.5.3 Theoretical sampling and saturation

Sampling decisions are guided by emerging categories rather than predetermined demographic quotas. For example, if early analysis suggests that tool friction is a major driver of negative modelling experiences, additional participants may be recruited from courses with different tool ecosystems to explore variation. Data collection continues until categories are sufficiently elaborated and additional data no longer contribute new properties or relationships to the explanatory account [1, 24].

4.5.4 Bloom's taxonomy as a sensitising concept

Bloom's revised taxonomy is used as a sensitising concept to support the interpretation and communication of findings, particularly when describing the cognitive character of reported modelling challenges (e.g., difficulties related to understanding constructs and their semantics, applying modelling procedures, and evaluating model adequacy) [14]. In practice, Bloom is applied *after* grounded categories have been developed: codes and categories are generated inductively, and Bloom-inspired terminology is used selectively as an analytic lens and reporting vocabulary where it aligns with participant accounts and helps articulate differences between types of challenges across modelling activities. Bloom is therefore not used as a coding scheme or set of predefined bins; this avoids forcing data into a priori frameworks and is consistent with grounded theory guidance for maintaining the primacy of emergent concepts [24].

4.6 Rigour and trustworthiness

To ensure rigour, the study follows established guidance for conducting grounded theory in software engineering [24] and adopts a socio-technical stance consistent with STGT [11]. Trustworthiness is addressed through practices that support credibility, dependability, confirmability, and transferability [11, 24].

Credibility is supported by grounding categories in concrete, experience-near accounts and by using activity- or artefact-centred elicitation to anchor participant reflections in specific modelling tasks and decisions. Constant comparison is used throughout analysis to refine categories, identify variation across participants and contexts, and seek disconfirming cases that challenge emerging interpretations. Where feasible, preliminary interpretations are discussed with supervisors and refined through peer debriefing.

Dependability and confirmability are supported through a transparent analytic trail. Coding decisions, category evolution, and sampling changes are documented through memos, and links between claims and supporting excerpts are preserved. Iteration between data collection and analysis is recorded to make the development of the explanatory account traceable.

Transferability is addressed by providing contextual detail about participants, course settings, modelling notations, and tool environments, enabling readers to assess the relevance of findings to other educational contexts. Rather than aiming for statistical generalisation, the thesis targets analytical generalisation by describing patterns of challenges and support needs and the conditions under which they appear.

4.7 Ethics and data management

Participation is voluntary and based on informed consent. Interview recordings and transcripts are anonymised, and any potentially identifying details (e.g., course identifiers, project names, personal data) are removed or generalised. Data are stored securely and used only for the purposes of this research. When collecting modelling artefacts (e.g., student models or assignments), additional care is taken to remove identifying information and to ensure that artefacts are used in ways consistent with participants' consent and institutional policies.

4.8 Methodological limitations

As a qualitative, context-sensitive study, the findings aim for analytical rather than statistical generalisation. Participant accounts may be influenced by recall bias, social desirability, and variation in course contexts. Additionally, tool- and notation-specific experiences may limit transferability across institutions or modelling families. These limitations are mitigated by triangulating accounts with concrete artefacts where possible, sampling

for variation as categories emerge, and documenting contextual factors that shape the observed phenomena [24, 11].

5 Work plan

5.1 Work plan

Figure 5.1 summarises the planned workflow of this thesis, from preparation and study design to data collection, grounded-theory analysis, and writing.

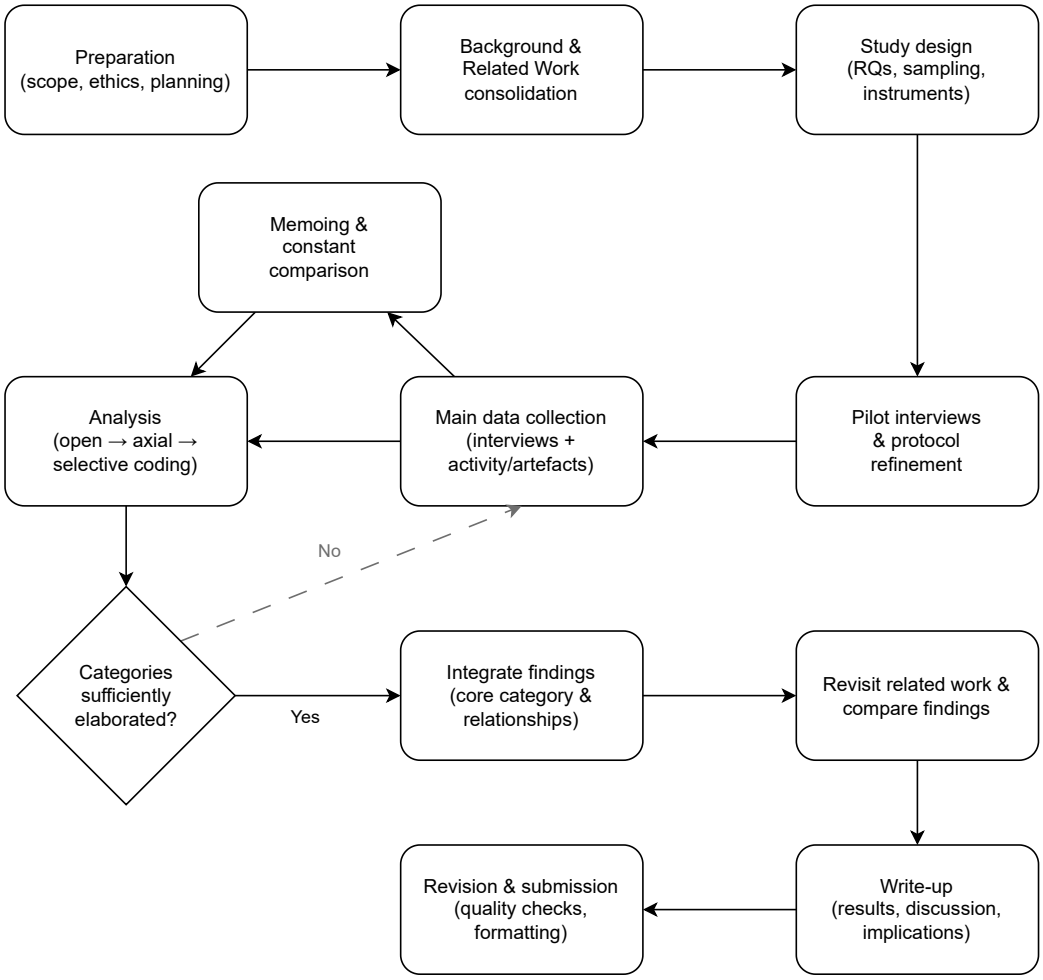

Figure 5.1: Planned thesis workflow (work plan).

The workflow is intentionally iterative. After piloting, data collection and analysis proceed in parallel: interview prompts and sampling can be adjusted as categories emerge, and memoing supports continuous refinement of concepts. Iteration continues until categories are sufficiently elaborated to integrate an explanatory account, which is then consolidated into the final write-up and revisions.

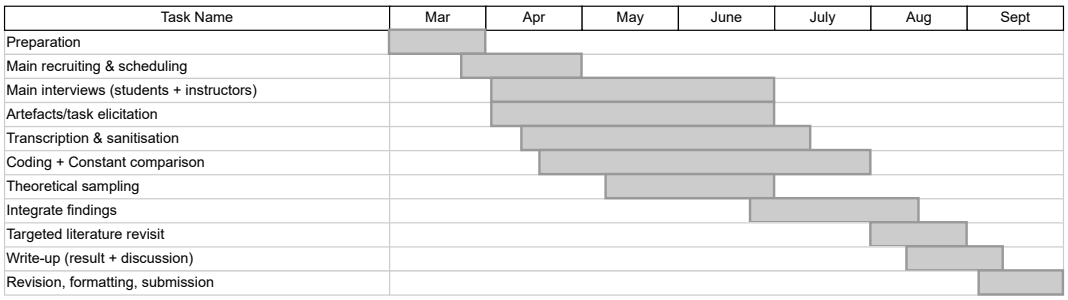

Figure 5.2: Gantt chart of planned activities.

Activity descriptions.

- **Preparation:** Finalise scope, instruments plan, consent and data handling procedures; obtain required approvals; refine interview guides for students and instructors; define prompts for task/artefact-anchored elicitation; adjust wording, ordering, timing, and probing strategy; refine the task/artefact elicitation flow.
- **Recruiting & scheduling:** Recruit student and instructor participants; schedule sessions; manage consent and logistics.
- **Main interviews (students + instructors):** Conduct semi-structured interviews to elicit challenges, strategies/support needs, and socio-technical shaping factors.
- **Artefact/task elicitation:** Collect or walk through concrete models/tasks to ground accounts in situated examples (where feasible).
- **Transcription & sanitisation:** Transcribe interviews; anonymise; prepare excerpts/artefact notes for analysis.
- **Coding & constant comparison:** Perform iterative coding and memoing; compare across participants/courses/tools to refine categories.
- **Theoretical sampling (as needed):** Recruit additional participants or explore different contexts to elaborate emerging categories and test variation.
- **Integrate findings:** Consolidate categories around a core explanatory account and key relationships.
- **Targeted literature revisit:** Re-engage with the most relevant literature to position findings, highlight alignments/divergences, and sharpen implications.
- **Write-up:** Draft results, discussion, and implications/recommendations; connect findings to RQs.
- **Revision & submission:** Quality checks, formatting, supervisor feedback incorporation, final edits, submission.

Bibliography

[1] S. Adolph, W. Hall, and P. Kruchten. "Using Grounded Theory to Study the Experience of Software Development". In: *IEEE Software* 28.4 (2011), pp. 24–27 (cit. on pp. 10, 21–25).

[2] D. Bogdanova and M. Snoeck. "CaMeLOT: An Educational Framework for Conceptual Data Modelling". In: *Information and Software Technology* 110 (2019), pp. 92–107. DOI: 10.1016/j.infsof.2019.02.006 (cit. on pp. 1, 5, 9, 12, 13, 16, 17, 21).

[3] D. Bogdanova and M. Snoeck. "Domain Modelling in Bloom: Deciphering How We Teach It". In: *The Practice of Enterprise Modeling (PoEM 2017)*. Vol. 305. Lecture Notes in Business Information Processing. 2017, pp. 3–17. DOI: 10.1007/978-3-319-70241-4_1 (cit. on pp. 1, 5, 6, 9, 12, 13, 16, 18, 21).

[4] D. Bork. "A Framework for Teaching Conceptual Modeling and Metamodeling Based on Bloom's Revised Taxonomy of Educational Objectives". In: *Proceedings of the 52nd Hawaii International Conference on System Sciences (HICSS)*. 2019 (cit. on pp. 1, 5, 7, 9, 12, 13, 16, 18, 21).

[5] R. A. Buchmann et al. "Conceptual Modelling in Education: A Position Paper". In: *CEUR Workshop Proceedings*. 2019 (cit. on pp. 1, 4, 5).

[6] CC2020 Task Force. *Computing Curricula 2020: Paradigms for Global Computing Education*. Association for Computing Machinery and IEEE Computer Society, 2020. DOI: 10.5555/3467967.C5641384 (cit. on p. 12).

[7] R. Chakraborty and G. Liebel. ""We Do Not Understand What It Says": Studying Student Perceptions of Software Modeling". In: *Proceedings of the 25th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings (MODELS-C)*. CEUR-WS, 2022. URL: https://ceur-ws.org/Vol-3418/paper06.pdf (cit. on pp. 1, 9).

[8] K. Charmaz. *Constructing Grounded Theory: A Practical Guide through Qualitative Analysis*. London: Sage Publications, 2006 (cit. on p. 10).

[9] B. G. Glaser and A. L. Strauss. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Chicago, IL: Aldine Publishing Company, 1967 (cit. on p. 10).

[10] I. Graßl et al. “Domain Diversity, Motivation, Inclusion, and Feedback in Software Modelling Education”. Manuscript submitted to ACM FSE 2026 (SEET track). 2026 (cit. on pp. 1, 8, 9, 13, 14, 16, 20, 21).

[11] R. Hoda, J. Noble, and S. Marshall. “Socio-Technical Grounded Theory for Software Engineering”. In: *IEEE Transactions on Software Engineering* 44.12 (2018), pp. 1208–1226 (cit. on pp. 2, 11, 21, 22, 25–27).

[12] R. Holmes, M. Allen, and M. Craig. “Dimensions of Experientialism for Software Engineering Education”. In: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training Track (ICSE-SEET '18)*. New York, NY, USA: ACM, 2018. DOI: 10.1145/3183377.3183380 (cit. on pp. 9, 14–16, 20, 21).

[13] B. Kitchenham and S. L. Pfleeger. “Teaching Survey Research in Software Engineering”. In: *ACM SIGSOFT Software Engineering Notes* 33.6 (2008), pp. 1–10.

[14] D. R. Krathwohl. “A Revision of Bloom’s Taxonomy: An Overview”. In: *Theory Into Practice* 41.4 (2002), pp. 212–218 (cit. on pp. 2, 6, 11, 12, 17, 25).

[15] T. Kühne. “What is a Model?” In: *Language Engineering for Model-Driven Software Development*. Ed. by J. Bézivin and R. Heckel. Vol. 4101. Dagstuhl Seminar Proceedings (DagSemProc). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2005, pp. 1–10. DOI: 10.4230/DagSemProc.04101.15 (cit. on p. 4).

[16] T. Li et al. “Understanding the Challenges and Requirements for Facilitating iStar Learning: An Empirical Study with iStar Learners”. In: *Information and Software Technology* 187 (2025), p. 107839. DOI: 10.1016/j.infsof.2025.107839 (cit. on pp. 1, 2, 7, 9, 18, 19, 21–25).

[17] G. Liebel, O. Badreddin, and R. Heldal. “Model Driven Software Engineering in Education: A Multi-Case Study on Perception of Tools and UML”. In: *2017 IEEE 30th Conference on Software Engineering Education and Training (CSEE&T)*. 2017, pp. 124–133. DOI: 10.1109/CSEET.2017.29 (cit. on pp. 1, 2, 8, 9, 15, 16, 19, 21).

[18] G. Liebel et al. “Human factors in model-driven engineering: future research goals and initiatives for MDE”. In: *Software and Systems Modeling* 23 (2024), pp. 801–819. DOI: 10.1007/s10270-024-01188-8 (cit. on pp. 8, 9, 16, 20).

[19] J. Linaker, P. Runeson, and B. Regnell. “Challenges in Survey Research”. In: *Empirical Software Engineering* 20.4 (2015), pp. 924–962.

[20] J. M. Lourenço. *The NOVAthesis LaTeX Template User’s Manual*. NOVA University Lisbon. 2021. URL: https://github.com/joaomlourenco/novathesis/raw/main/template.pdf.

[21] I. Maslov. "Towards Empirically Validated Process Modelling Education Using a BPMN Formalism". In: *Research Challenges in Information Science (RCIS 2022)*. Vol. 446. Lecture Notes in Business Information Processing. Springer, 2022, pp. 803–810. DOI: 10.1007/978-3-031-05760-1_58 (cit. on pp. 1, 7, 9, 19, 21).

[22] I. Maslov, S. Poelmans, and K. Rosenthal. "Conceptual Modeling Education: A Bibliometric Literature Review and Avenues for Future Research". In: *Business & Information Systems Engineering* (2025), pp. 1–33. DOI: 10.1007/s12599-025-00930-w (cit. on pp. 1, 2, 4–6, 17, 21).

[23] Y. Sedelmaier and D. Landes. "A Research Agenda for Identifying and Developing Required Competencies in Software Engineering". In: *International Journal of Engineering Pedagogy (iJEP)* 3.2 (2013), pp. 30–35. DOI: 10.3991/ijep.v3i2.2448 (cit. on pp. 5, 8, 9, 13, 15, 16, 21).

[24] K.-J. Stol, P. Ralph, and B. Fitzgerald. "Grounded Theory in Software Engineering Research: A Critical Review and Guidelines". In: *IEEE Transactions on Software Engineering* 42.12 (2016), pp. 1201–1222 (cit. on pp. 2, 10, 21–27).

[25] A. Strauss and J. Corbin. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory*. 2nd ed. Thousand Oaks, CA: Sage Publications, 1998 (cit. on p. 10).